

Find Median in a Stream.

Median -
is the middle
value in an
ordered dataset.

[7 | 11 | 4 | 9 | 15 | 2 | 1 | 18]

Unordered	Ordered	Median
7	7	7
7 11	7 11	9
7 11 4	4 7 11	7
7 11 4 9	4 7 9 11	8
7 11 4 9 15	4 7 9 11 15	9
}	}	
}	}	

return this vector
as ans.

for 7 11 4 $\Rightarrow$ 4 7 11
& for 7 11 4 9
 $\Rightarrow$ 4 7 9 11
 $\frac{(7+9)}{2} = 8$

Approach 2:

We basically want our
data to be in sorted form
& when the new ele
comes, we can simply
insert into our DS
without disturbing the
sorted order.

1st: Brute-force Approach:

[2 | 4 | 6 | 8 | 10 | 12 | 14] Now if 7 comes
[2 | 4 | 6 | 7 | 8 | 10 | 12 | 14]

Either, with every coming element,

i) We sort the arr & find the median, T.C: $N(N \log N)$
OR

ii) we place the new ele to it's
correct position to maintain
the sorted order.

$$N * (N)$$

for shifting

$$T.C: O(N^2)$$

$$T.C \approx O(N^2 \log N)$$

Median =

The idea is to divide the data into
2 equal halves within minh & maxh
in a way, their top element
will be the median.

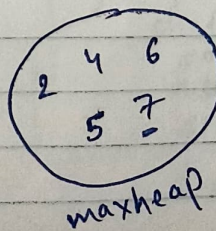
$$\text{minheap.size() == maxheap.size()} \quad \frac{7+8}{2}$$

$$\text{median} = \frac{\text{topele(minh)} + \text{topele(maxh)}}{2}$$

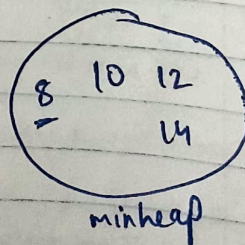
& if $\text{maxheap.size()} > \text{minheap.size()}$ or

$$\text{maxheap.size()} = \text{minheap.size()} + 1$$

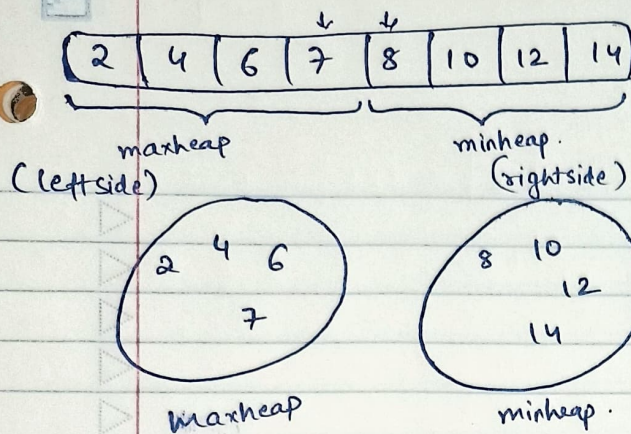
$$\text{median} = \text{topele(maxh)} \Rightarrow 7$$



maxheap



minheap



3 functions to complete.

```
// func to insert heap.
void insertHeap(int &x) {
    ...
}

// function to balance heaps
void balanceHeaps() {
    ...
}

// function to return median
double getMedian() {
    ...
}
```

this 2 will be called by compiler.

// Declare 2 heaps at the top as it will be used by all 3 funcs.

```
public:
priority_queue<int> LeftMaxHeap;
priority_queue<int, vector<int>, greater<int>> RightMinHeap;
```

Pseudocode:

① Leftside (Maxheap) == Rightside (Minheap)
 median = $\frac{\text{top} + \text{top}}{2}$

② Leftside - 1 == Rightside
 median = Leftside (top)

③ Rightside > Leftside
 pop from rightside (Minheap)
 & push into leftside (Maxheap)
 until
 Leftside == rightside
 or
 Leftside - 1 == rightside

if (Leftside - 1 > Rightside)
 pop from leftside
 & push into rightside

```
// Function to insert heap.
void InsertHeap(int &x) {
    // Base case
    if (LeftMaxHeap.empty()) {
        LeftMaxHeap.push(x);
        return;
    }
    if (x > LeftMaxHeap.top())
        RightMinHeap.push(x);
    else
        LeftMaxHeap.push(x);
    balanceHeaps();
}
```

```
// Function to balance heaps.
void balanceHeaps() {
    // MinHeap(Right) size should not be greater than MaxHeap
    if (RightMinHeap.size() > LeftMaxHeap.size()) {
        LeftMaxHeap.push(RightMinHeap.top());
        RightMinHeap.pop();
    }
    else {
        // only if the difference b/w
        // left-right > 1
        if (RightMinHeap.size() < LeftMaxHeap.size() - 1) {
            RightMinHeap.push(LeftMaxHeap.top());
            LeftMaxHeap.pop();
        }
    }
}
```

	Left	right
Left	5	4
right	6	4

```
// Func to return Median
double getMedian() {
```

```
    // Left > right
    if (LeftMaxHeap.size() > RightMinHeap.size())
        return LeftMaxHeap.top();
}
```

```
    else {
        double ans = (top(MinH) + top(MaxH));
        ans /= 2;
        return ans;
    }
}
```